INFORME FINAL

Migración de Base de Datos Employees desde MariaDB hacia PostgreSQL

Tecnología de Base de Datos I

---

1. Portada

Título: Migración, Verificación y Modelado de la Base de Datos Employees utilizando MariaDB, PostgreSQL y Pgloader.

Estudiante: Dorian Escobar

Materia: Tecnologia de Base de Datos 1

Docente: ING. Jared Lopez Leaño

Fecha: 29 de septiembre de 2026

---

2. Introducción

El presente informe documenta el proceso completo de migración de la base de datos employees desde MariaDB hacia PostgreSQL.

La actividad consistió en preparar el entorno mediante Docker, analizar la estructura de la base origen, realizar la migración utilizando pgloader, adaptar los elementos incompatibles entre ambos gestores y verificar que la información migrada mantenga la integridad y consistencia esperada.

Como parte del proceso también se realizó la creación y validación de vistas en PostgreSQL, además de la generación del modelo físico utilizando PgModeler.

---

3. Objetivos

Objetivo general

Migrar correctamente la base de datos employees desde MariaDB hacia PostgreSQL, documentando cada etapa del proceso, realizando adaptaciones necesarias y verificando la integridad de la información.

Objetivos específicos

• Configurar el ambiente de trabajo mediante Docker.

• Analizar la estructura de la base de datos MariaDB.

• Crear la base destino en PostgreSQL.

• Ejecutar la migración utilizando pgloader.

• Adaptar tipos de datos incompatibles.

• Recrear y probar vistas en PostgreSQL.

• Verificar registros, relaciones y consistencia de datos.

• Generar documentación técnica del proceso.

---

4. Herramientas utilizadas

Docker

Utilizado para administrar los servicios necesarios:

• MariaDB.

• PostgreSQL.

• Adminer.

• pgAdmin.

MariaDB 11.8.9

Gestor utilizado como base de datos origen.

PostgreSQL 18.6

Gestor utilizado como base de datos destino.

Pgloader

Herramienta utilizada para realizar la migración automática entre

MariaDB y PostgreSQL.

Permite:

• Crear tablas.

• Migrar datos.

• Crear índices.

• Convertir tipos de datos.

PgModeler

Utilizado para representar gráficamente el modelo físico de PostgreSQL.

pg_dump

Utilizado para generar el respaldo final de la base migrada.

---

5. Arquitectura implementada

La arquitectura utilizada fue:

MariaDB

|

|

pgloader

|

|

PostgreSQL

|

|

PgModeler

Descripción:

• MariaDB contiene la base original `employees`.

• Pgloader realiza la transferencia y conversión.

• PostgreSQL almacena la información migrada.

• PgModeler permite documentar visualmente la estructura final.

---

6. Preparación del entorno

Los servicios fueron levantados mediante Docker.

Comando utilizado:

docker compose up -d

Verificación:

docker compose ps

Servicios activos:

adminer Up

mariadb Up

pgadmin4 Up

postgresql Up (healthy)

Esto confirmó que el entorno estaba preparado para realizar la

migración.

---

7. Análisis de la base origen MariaDB

Se realizó la conexión:

mariadb -h 127.0.0.1 -u root -p

Verificación de versión:

SELECT VERSION();

Resultado:

11.8.9-MariaDB

Base utilizada:

employees

7.1 Tablas identificadas

Consulta:

SHOW TABLES;

Resultado:

current_dept_emp

departments

dept_emp

dept_emp_latest_date

dept_manager

employees

salaries

titles

7.2 Vistas identificadas

Consulta:

```
SELECT TABLE_NAME
FROM information_schema.views
WHERE table_schema='employees';
```

Resultado:

current_dept_emp

dept_emp_latest_date

---

8. Creación de estructura PostgreSQL

Se creó la base destino:

psql -h 127.0.0.1 -U dorian -d postgres \

-c "CREATE DATABASE pdb_employees;"

Resultado:

CREATE DATABASE

---

9. Configuración de pgloader

Archivo utilizado:

migracion.load

Configuración principal:

LOAD DATABASE

FROM mysql://root:123123@localhost:3306/employees

INTO postgresql://dorian:1251*@localhost:5432/pdb_employees

WITH

include drop,

create tables,

create indexes,

reset sequences,

workers = 8,

concurrency = 2,

batch rows = 10000

CAST

type datetime to timestamp,

type enum to text;

Explicación de configuración

include drop

Elimina objetos existentes antes de iniciar la migración para evitar

conflictos.

create tables

Genera automáticamente las tablas en PostgreSQL.

create indexes

Crea los índices necesarios para mantener el rendimiento.

reset sequences

Ajusta las secuencias internas después de cargar información.

workers y concurrency

Permiten utilizar procesos paralelos para mejorar el tiempo de

migración.

batch rows

Define la cantidad de registros procesados por lote.

CAST

Permite convertir tipos incompatibles entre gestores.

---

10. Ejecución de la migración

Comando utilizado:

pgloader migracion.load

Resultado:

Total import time

✓ 3919015 rows

134.9 MB

La migración finalizó correctamente sin errores.

---

11. Adaptaciones realizadas MariaDB → PostgreSQL

Durante la migración fue necesario adaptar algunos tipos de datos.

Conversión ENUM

MariaDB:

gender ENUM('M','F')

PostgreSQL:

gender TEXT

Motivo:

Se realizó la conversión para garantizar compatibilidad durante el proceso automático de migración.

Conversión numérica

MariaDB:

INT(11)

PostgreSQL:

BIGINT

Esta adaptación permitió conservar correctamente los identificadores y valores numéricos.

---

12. Migración y adaptación de vistas

Las vistas originales no fueron trasladadas automáticamente por

pgloader, por lo cual fueron recreadas manualmente en PostgreSQL.

Vista current_dept_emp

Descripción:

Muestra los empleados que actualmente poseen una asignación vigente

dentro de un departamento.

Script:

```
CREATE OR REPLACE VIEW employees.current_dept_emp AS
SELECT
de.emp_no,
de.dept_no,
de.from_date,
de.to_date
FROM employees.dept_emp de
WHERE CURRENT_DATE BETWEEN de.from_date AND de.to_date;
```

Vista dept_emp_latest_date

Descripción:

Obtiene la última fecha registrada de asignación departamental por

empleado.

Script:

```
CREATE OR REPLACE VIEW employees.dept_emp_latest_date AS
SELECT
emp_no,
MAX(from_date),
MAX(to_date)
FROM employees.dept_emp
GROUP BY emp_no;
```

---

13. Verificación de vistas

Consulta:

```
\dv employees.*
```

Resultado:

current_dept_emp

dept_emp_latest_date

Prueba de funcionamiento:

SELECT *

FROM employees.current_dept_emp

LIMIT 5;

Resultado:

10001 | d005 | 1986-06-26 | 9999-01-01

10002 | d007 | 1996-08-03 | 9999-01-01

La vista devuelve información correctamente.

---

14. Verificación de registros migrados

Se realizaron conteos en PostgreSQL.

Resultados:

Tabla Registros

---------- ----------

employees 300024

departments 9

dept_emp 331603

dept_manager 24

salaries 2844047

titles 443308

---

15. Comparación MariaDB vs PostgreSQL

Se realizaron los mismos conteos en la base origen y destino.

Resultado:

Tabla MariaDB PostgreSQL Estado

--- --- --- ---

employees 300024 300024 Correcto

departments 9 9 Correcto

dept_emp 331603 331603 Correcto

dept_manager 24 24 Correcto

salaries 2844047 2844047 Correcto

titles 443308 443308 Correcto

Análisis:

Los resultados coinciden en todas las tablas verificadas, demostrando

que no existió pérdida de información durante la migración.

---

16. Verificación de integridad referencial

Se realizaron pruebas buscando registros sin relación.

Empleados sin departamento

Resultado:

empleados_sin_departamento = 0

Departamentos sin empleados

Resultado:

departamentos_sin_empleados = 0

Análisis:

La información mantiene correctamente las relaciones entre empleados y

departamentos.

---

17. Verificación mediante checksum

Como parte de la validación de la migración se realizaron comprobaciones mediante **checksum MD5**, una técnica utilizada para generar una huella digital única a partir de los datos almacenados en una tabla.

El objetivo de esta verificación es comprobar que la información migrada desde MariaDB hacia PostgreSQL mantenga la misma secuencia de registros y no haya sufrido modificaciones, pérdidas o alteraciones durante el proceso de transferencia.

Para generar el checksum se tomó como referencia el identificador principal de los registros (emp_no) y se aplicó una función MD5 sobre la información ordenada. De esta manera, si los datos del origen y destino son iguales, el resultado generado debe coincidir.

La conexión utilizada fue:

psql -h 127.0.0.1 -U dorian -d pdb_employees

Tabla employees

Se realizó la comprobación sobre la tabla principal de empleados, verificando la cantidad total de registros y generando su valor de checksum.

Consulta ejecutada:

SELECT

COUNT(*) AS registros,

md5(string_agg(emp_no::text, ',' ORDER BY emp_no)) AS checksum

FROM employees.employees;

Descripción del procedimiento:

COUNT(*) permite verificar la cantidad total de registros existentes en la tabla.

string_agg() genera una cadena con todos los identificadores de empleados.

ORDER BY emp_no mantiene un orden estable de los datos para evitar diferencias en el cálculo.

md5() genera una huella digital única del conjunto de registros analizados.

Resultado obtenido:

Registros:

300024

Checksum:

7daff798dded7e6b829f63c04e5869c2

El resultado confirma que los identificadores de los empleados migrados mantienen la misma estructura y cantidad de registros esperados después de la migración.

Tabla dept_emp

También se realizó la verificación sobre la tabla dept_emp, la cual contiene la relación entre empleados y departamentos.

La segunda validación fue aplicada sobre la tabla:

dept_emp

Esta tabla es importante porque representa la relación entre empleados y departamentos dentro del modelo de datos.

Consulta ejecutada:

SELECT

COUNT(*) AS registros,

md5(string_agg(emp_no::text, ',' ORDER BY emp_no)) AS checksum

FROM employees.dept_emp;

Resultado obtenido:

Registros:

331603

Checksum:

005dea57071c11f9f7f407dbd4c84444

Esta comprobación permite validar que las relaciones entre empleados y departamentos fueron transferidas correctamente, manteniendo la información necesaria para conservar la integridad del modelo.

Análisis de resultados

Al comparar estos resultados con los conteos obtenidos previamente en MariaDB y PostgreSQL, se comprobó que las tablas verificadas mantienen la misma cantidad de registros esperada.

La validación mediante checksum complementa las pruebas realizadas anteriormente:

Conteo de registros por tabla.

Comparación entre base origen MariaDB y base destino PostgreSQL.

Verificación de integridad referencial.

Prueba de funcionamiento de vistas.

La coincidencia de la cantidad de registros y la generación de valores de comprobación permiten confirmar que los datos críticos de las tablas principales fueron conservados durante el proceso de migración desde MariaDB hacia PostgreSQL.

---

18. Backup final

Se generó un respaldo PostgreSQL:

backup_pdb_employees.dump

Comando:

```
pg_dump \
-h 127.0.0.1 \
-U dorian \
-F c \
-d pdb_employees \
```

-f backup_pdb_employees.dump

Tamaño generado:

35 MB

---

19. Modelo físico PgModeler

El modelo fue generado mediante PgModeler.

Incluye:

• Entidades.

• Atributos.

• Relaciones.

• Claves primarias.

• Claves foráneas.

Archivo:

![archivo sql PgModeler](./modelo/pdb_employees_modelo.sql)

---

20. Conclusión

La migración de la base de datos employees desde MariaDB hacia

PostgreSQL fue realizada correctamente.

Se logró:

- Migrar tablas y registros.
- Adaptar tipos de datos incompatibles.
- Recrear y probar vistas.
- Validar conteos entre origen y destino.
- Comprobar integridad referencial.
- Generar evidencias mediante checksum.
- Crear respaldo y documentación técnica.

El proceso demuestra una migración completa, documentada y verificada utilizando herramientas adecuadas para administración y transferencia de bases de datos.